

Corso di Laurea in Informatica A.A. 2024-2025

Laboratorio di Sistemi Operativi

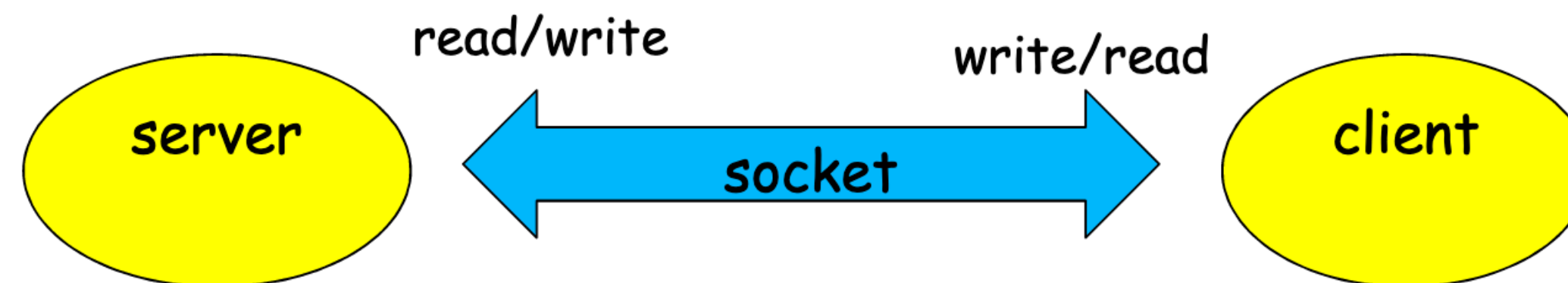
Alessandra Rossi

Le socket

- Definisce un canale di comunicazione bidirezionale
- Progettato per client-server
- Fornisce dei file descriptor (intero non negativo)
 - si possono usare read, write, close, etc.
- Vari tipi di socket
 - socket locali - stessa macchina
 - socket TCP/UDP - tramite rete

Panoramica

- Indipendenti dal linguaggio
- Spesso chiamate Berkley socket (BSD)
- Rappresentano un estremo della comunicazione
- Chiamati con `socket()`



Domini e Stili di Comunicazione

- I socket permettono di specificare il tipo di comunicazione attraverso le nozioni di **dominio** e di **stile**;
- Il **dominio** di un socket equivale alla scelta di una famiglia di protocolli. Le correnti release del kernel di molti Unix prevedono ventisei diverse famiglie;
- Ogni dominio è individuato univocamente da un intero non negativo, cui corrispondono una o più costanti simboliche del tipo **PF_nomefamiglia**, definite nell'header **<sys/socket.h>**;

Domini e Stili di Comunicazione

- Tra i domini più importanti si ricordano:
 - `PF_LOCAL` (o `PF_UNIX`, o `PF_FILE`), per le comunicazioni in locale tramite filesystem (reale o virtuale);
 - `PF_INET`, la famiglia TCP/IP con Ipv4;
 - `PF_INET6`, la famiglia TCP/IP con Ipv6;
 - `PF_IPX`, famiglia di protocolli per reti Novell;
 - `PF_APPLETALK`, famiglia di protocolli per reti Appletalk;
- A ciascun dominio possono corrispondere in teoria uno o più schemi di indirizzamento, ossia tipi di indirizzi, per cui i socket prevedono la nozione di **famiglia di indirizzi**, ciascuna individuata univocamente attraverso un numero non negativo, cui corrisponde (sempre tramite l'header `socket.h`) una costante simbolica del tipo `AF_nomefamiglia`;

Stili di Comunicazione

- Lo stile di comunicazione è individuato da costanti simboliche del tipo `SOCK_nomestile`, definite anch'esse nell'header `<sys/socket.h>`;
- Gli stili principali sono:
 - `SOCK_STREAM`, corrispondente ad un canale di trasmissione bidirezionale a flusso. con connessione. sequenziale ed affidabile;
 - `SOCK_DGRAM`, che consiste in una trasmissione a pacchetti (`datagram`) di lunghezza max. prefissata, senza connessione e non affidabile;
 - `SOCK_RAW`, per l'accesso a basso livello ai protocolli di rete ed alle varie interfacce;
- Assegnato un dominio, la scelta dello stile di comunicazione corrisponde in pratica ad individuare uno specifico protocollo tra quelli appartenenti al dominio;
- Non tutte le combinazioni “dominio–stile di comunicazione” sono valide, in quanto non è detto che in una famiglia di protocolli esista un elemento per ciascuno dei possibili stili;

Indirizzamento

- In UNIX le strutture dati per la gestione degli indirizzi sono progettate per essere adatte a diversi domini di comunicazione e protocolli. Ad es.

- `sa_family = AF_INET`
- `sa_family = PF_LOCAL, PF_UNIX`

- **Generico indirizzo:**

```
#include <sys/socket.h>
struct sockaddr {
    unsigned short sa_family;    // address family, AF_XXX
    char           sa_data[14]; // 14 bytes of protocol
};                             // address
```

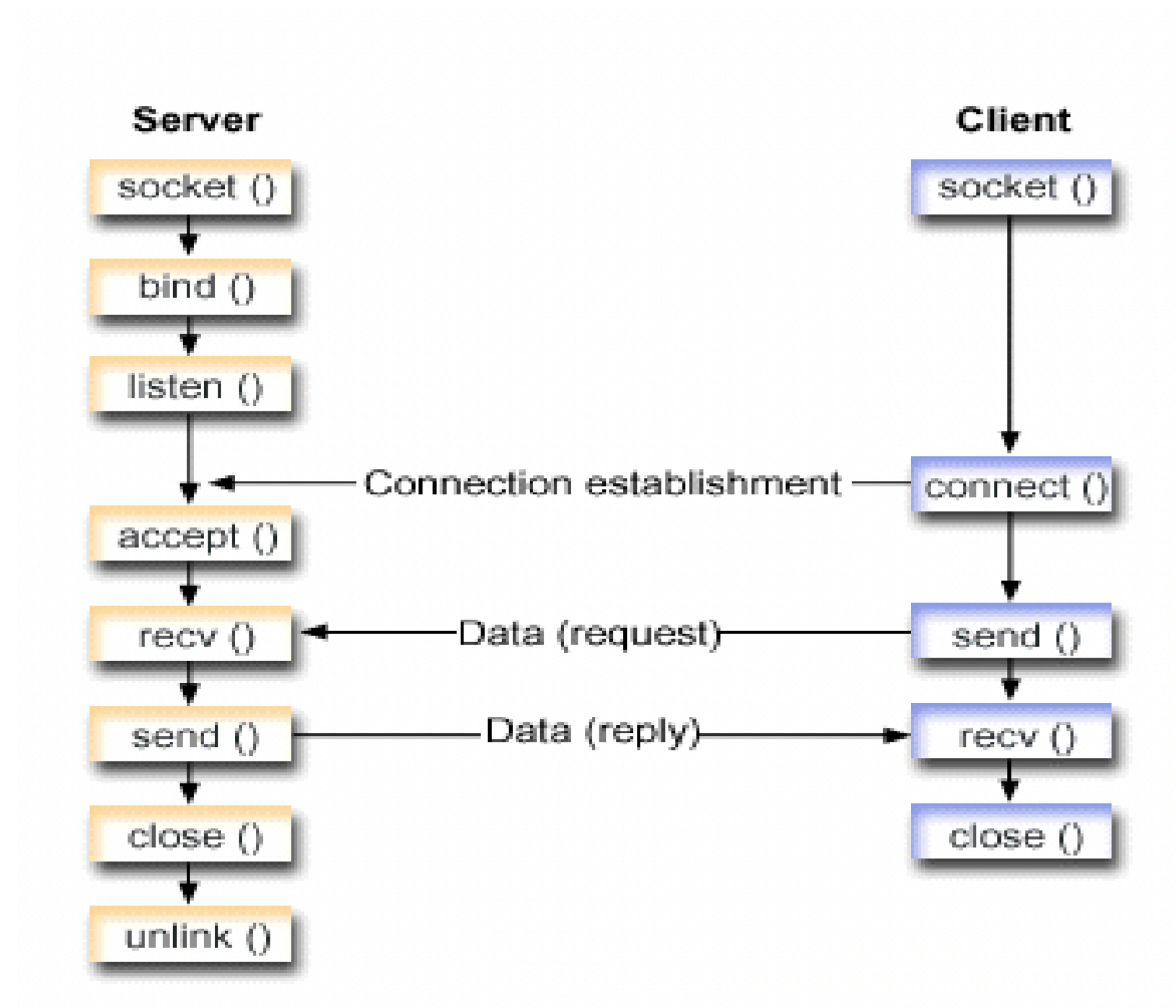

Indirizzi

- Siamo interessati a `sa_family = AF_INET`, cioè al dominio “internet”
- In tal caso servono 2 byte per un numero di porta e 4 byte per un indirizzo IP

```
#include <netinet/in.h>
struct sockaddr_in {
    short int sin_family;           // Address family
    { unsigned short int sin_port;  // Port number
      struct in_addr sin_addr;     // IP Internet Address
      unsigned char sin_zero[8];   // Stessi byte di...
    };                             // ...struct sockaddr
```

14 bytes

Flusso Socket Server-Client



Funzione Socket

```
#include <sys/socket.h>
```

```
int socket(int family, int type, int protocol);
```

- apre un socket, allocando una voce nella tabella dei file del kernel e permettendo la specifica del dominio, dello stile e del protocollo di comunicazione;
- *family* è la famiglia cui appartiene il protocollo (dominio);
- *type* definisce il tipo di comunicazione,
- *protocol* specifica il protocollo della famiglia;
- restituisce **-1** in caso di insuccesso, un numero **positivo** (il **socket descriptor**) altrimenti.

Creare un socket

```
int socket(int famiglia, int tipo, int protocollo);
```

- Crea un socket di una determinata categoria
- Nel nostro caso:
 - famiglia = PF_LOCAL oppure PF_INET
 - (PF = *Protocol Family*)
 - tipo = SOCK_STREAM
 - protocollo = 0
- Restituisce il descrittore del socket, oppure -1

Funzione Socket

Valori per *family*

(AF_) PF_UNIX
(AF_) PF_INET
(AF_) PF_INET6
(AF_) PF_APPLETALK
(AF_) PF_X25
...

Tipo di protocolli

Protocolli per comunicazioni locali su sistemi UNIX
Protocolli per Internet (vers. 4)
Protocolli per Internet (vers. 6)
Protocolli per LAN Appletalk
Protocolli dello standard ITU X.25 per reti a commutazione di pacchetti

Valori per *type*

SOCK_STREAM
SOCK_DGRAM
SOCK_RAW
SOCK_SEQPACKET
...

Tipo di trasmissione

a flusso, sequenziale ed affidabile
a pacchetti di lungh. max. fissata, non sequenziale e non affidabile
accesso a basso livello (costruzione "manuale" dei pacchetti)
a pacchetti di lungh. max. fissata, sequenziale ed affidabile

Valori per *protocol*

IPPROTO_TCP
IPPROTO_UDP
IPPROTO_RAW
IPPROTO_ICMP
...

Protocollo della famiglia INET (definiti in <netinet/in.h>)

TCP
UDP
IP
ICMP

Gli Include

- @ Un programma che usa socket deve includere:
 - @ `<sys/types.h>`
 - @ `<sys/socket.h>`

- @ Inoltre deve anche includere:
 - @ `<sys/un.h>` se usa socket del dominio:
 - @ `AF_LOCAL` o `AF_UNIX`, socket locali
 - @ `<netinet/in.h>` `<arpa/inet.h>` e `<arpa/netdb.h>` se usa socket del dominio:
 - @ `AF_INET`

Creare un socket locale

```
int fd = socket(PF_LOCAL, SOCK_STREAM, 0);  
  
if (fd<0) perror("socket"), exit(1);
```

```
int fd = socket(PF_INET, SOCK_STREAM, 0);  
  
if (fd<0) perror("socket"), exit(1);
```

```
int fd = socket(PF_INET, SOCK_DGRAM, 0);  
  
if (fd<0) perror("socket"), exit(1);
```

La funzione bind

```
#include <sys/socket.h>
```

```
int bind(int sd, const struct sockaddr *my_addr, int addrlen);
```

- assegna un indirizzo locale al (l'insieme di) socket individuato dal socket descriptor *sd*;
- *sd* è un socket descriptor ottenuto da una precedente chiamata a socket;
- *my_addr* è l'indirizzo locale, specificato secondo il formato caratteristico della famiglia di protocolli cui *sd* è riferito;
- *addrlen* è la lunghezza del suddetto indirizzo;
- restituisce **-1** in caso di errore, **0** altrimenti.

Panoramica: il server

Crea il socket	Fuzione socket
Gli assegna un indirizzo	Funzione bind
Si mette in ascolto	Funzione listen
Accetta nuove connessioni	Funzione accept
Chiude il socket	Funzione close
Cancella il file se socket locale	Funzione unlink

Funzione listen

```
#include <sys/socket.h>  
int listen (int sd, int backlog);
```

- utilizzata da un *server* in caso di comunicazione orientata alla connessione;
- pone il socket specificato da *sd* in modalità passiva, ossia in ascolto di eventuali connessioni, predisponendo per esso una coda per le connessioni in arrivo di lunghezza pari a *backlog*;
- *backlog* rappresenta il numero massimo di connessioni pendenti accettate. Se tale numero è superato, il client riceverà un errore, oppure – nel caso di protocolli come TCP che prevedono la ritrasmissione – la richiesta del client verrà ignorata in modo da poter essere ritentata;
- restituisce 0 in caso di successo, -1 altrimenti.

Accettare nuove connessioni

```
#include <sys/socket.h>
int accept (int sd, const struct sockaddr *addr, int addrlen);
```

- utilizzata da un **server** in caso di comunicazione orientata alla connessione, restituisce un nuovo socket descriptor su cui si potrà operare per effettuare la comunicazione con un client;
- estrae la prima connessione relativa al socket descriptor *sd* in attesa sulla coda delle connessioni, definita grazie ad una precedente chiamata a **listen** per *sd* ;
- nella struttura *addr* e nella variabile *addrlen* vengono restituiti, rispettivamente, l'indirizzo e la lunghezza di tale indirizzo per il client che si è connesso;
- restituisce un nuovo socket descriptor in caso di successo, **-1** altrimenti. Il nuovo socket eredita le caratteristiche di *sd*;

Struttura di un server

```
int fd1, fd2;
struct sockaddr_un mio_indirizzo;

mio_indirizzo.sun_family = AF_LOCAL;
strcpy(mio_indirizzo.sun_path, "/tmp/mio_socket");

fd1 = socket(PF_LOCAL, SOCK_STREAM, 0); //crea socket locale

bind(fd1, (struct sockaddr*)&mio_indirizzo,
      sizeof(mio_indirizzo));

listen(fd1, 5); //5 è la dim della coda di attesa

fd2 = accept(fd1, NULL, NULL);
...
close(fd2);
close(fd1);
unlink("/tmp/mio_socket");
```


Lato Client

Crea il socket	Funzione socket (uguale a quella server)
Si connette ad un server	Funzione connect
Chiude il socket	Funzione close

Funzione connect

```
#include <sys/socket.h>  
int connect (int sd, const struct sockaddr *serv_addr, int addrlen);
```

- utilizzata dal lato **client**, ha due funzionalità differenti a seconda che la comunicazione sia orientata o meno connessione;
- In entrambi i casi collega il socket locale di identificativo *sd* al socket remoto di indirizzo *serv_addr*;
- nel caso di comunicazioni con connessione attiva la procedura di avvio della connessione (il *three-way handshake* per il TCP) e ritorna solo quando la connessione è stabilita o si è verificato un errore;
- restituisce **0** in caso di successo, **-1** altrimenti.

Funzione close

```
#include <unistd.h>
```

```
int close(int sock_fd);
```

- restituisce zero in caso di successo, -1 in caso di errore
- serve per dichiarare che non si vuole più utilizzare il socket
- più processi dello stesso host possono condividere la stessa socket
 - solo se tutti avranno eseguito una `close()` il sistema operativo provvederà a chiudere la connessione (necessariamente di tipo `SOCK_STREAM`) concludendo il protocollo TCP
- la chiusura è simmetrica: la connessione sarà effettivamente chiusa quando sarà stata chiusa sia sul server che sul client

Struttura di un client

```
int fd;
struct sockaddr_un indirizzo;

indirizzo.sun_family = AF_LOCAL;
strcpy(indirizzo.sun_path, "/tmp/mio_socket");

fd = socket(PF_LOCAL, SOCK_STREAM, 0); // crea socket locale
connect(fd, (struct sockaddr*)&indirizzo,
        sizeof(indirizzo));

...
close(fd);
```

Leggere da un socket

- Si può usare read
- Se non ci sono dati da leggere, read **blocca** il processo in attesa di dati (come per una pipe)
- E' normale ottenere meno bytes di quelli richiesti (come per una pipe)
- Ottenere 0 bytes significa che il socket è vuoto ed inoltre è stato chiuso

Scrivere su un Socket

- Si può usare write
- E' normale riuscire a scrivere meno bytes di quelli richiesti (bisogna riprovare con il resto!)
- Se il socket è stato chiuso, il processo riceve il segnale SIGPIPE
 - di default, questo segnale termina il processo
 - se si ignora questo segnale (signal(SIGPIPE, SIG_IGN)), write restituisce -1 e imposta errno=EPIPE
 - oppure, si può catturare il segnale

Socket TCP/IP

Specificare indirizzi IP

- Usando la notazione *dotted* (puntata)

```
struct sockaddr_in indirizzo;  
  
if (inet_aton("143.225.5.3", &indirizzo.sin_addr) == 0)  
    perror("inet_aton"), exit(1);
```

- **inet_aton** (*ascii to network*)
- riempie direttamente una struttura **in_addr**
- restituisce 0 in caso di errore!

Impostare indirizzo

```
struct sockaddr_in my_addr;  
  
// host byte order  
my_addr.sin_family = AF_INET;  
  
// short, network byte order  
my_addr.sin_port = htons(MYPORT);  
  
// long, network byte order  
inet_aton("10.12.110.57", &(my_addr.sin_addr));  
  
// a zero tutto il resto  
memset(&(my_addr.sin_zero), '\0', 8);
```

Indirizzi TCP/IP per il server (bind)

- Il server chiama bind per stabilire su quale indirizzo mettersi in ascolto
- Di solito, il server sceglie solo la porta
- Come indirizzo IP, sceglie INADDR_ANY, così accetta connessioni dirette a qualunque indirizzo (uno stesso host può avere più indirizzi IP)

```
struct sockaddr_in mio_indirizzo;  
  
mio_indirizzo.sin_family      = AF_INET;  
mio_indirizzo.sin_port        = htons(5200);  
mio_indirizzo.sin_addr.s_addr = htonl(INADDR_ANY);  
  
bind(fd, (struct sockaddr *) &mio_indirizzo, sizeof(mio_indirizzo));
```

Indirizzi TCP/IP per il client (connect)

- Il client deve conoscere l'indirizzo IP e la porta del processo server

```
struct sockaddr_in indirizzo;  
  
indirizzo.sin_family = AF_INET;  
indirizzo.sin_port   = htons(5200);  
inet_aton("143.225.5.3", &indirizzo.sin_addr);  
  
connect(fd, (struct sockaddr *) &indirizzo, sizeof(indirizzo));
```


Trasmissione Dati

```
#include <unistd.h>
ssize_t write(int fd,
              const void *buf,
              size_t count);
```

- invia il contenuto del buffer `buf` al socket specificato
- si usa esclusivamente con `SOCK_STREAM`
- restituisce il numero di byte inviati oppure -1 in caso di errore
- è la stessa funzione che consente la scrittura su file

Ricezione

```
#include <unistd.h>
ssize_t read(int fd,
             void *buf,
             size_t count);
```

- solo per socket connessi (SOCK_STREAM)
- legge un messaggio di lunghezza massima `len` dal socket
- se non c'è alcun messaggio, il programma rimane sospeso
 - chiamata bloccante
- la funzione ritorna il numero di byte letti, `-1` in caso di errore
- è la stessa funzione che consente la lettura da un file

Leggere scrivere su socket

Stessa interfaccia per scrittura su file:

- Si possono passare le sd a processi figli che possono non distinguere tra sd e fd
- Possono essere usati anche con processi che lavorano su file locali.
- Esistono altre funzioni specifiche per leggere e scrivere su socket:
3 modalità di send, 3 modalità di receive.

Leggere e Scrivere su socket

-Send & Recv:

-send è come write (richiede connessione stabilita), ma ha flag per specificare modalità di scrittura,

-sendto permette la scrittura su connectionless socket,

-sendmsg per specificare buffer multipli

-recv come read con flag,

-recvfrom per ottenere indirizzo della fonte,

-rcvmsg per ricevere da buffer multipli

send() e sendto()

```
#include <sys/types.h>
#include <sys/socket.h>
int send(int s,
        const void *msg, int len,
        unsigned int flags);
int sendto(int s, const void *msg, int len,
          unsigned int flags,
          const struct sockaddr *to, int tolen);
```

man 2 send

- send() può essere utilizzata solo se s è stato connesso; sendto() sempre perché richiede di specificare l'indirizzo di destinazione
- restituisce -1 in caso di errore oppure il numero di byte effettivamente trasmessi
- flags si può lasciare a zero

sendTo()

```
#include <sys/types.h>
#include <sys/socket.h>
int sendto(int socket, const void *msg, int len,
           unsigned int flags,
           const struct sockaddr *to, int tolen);
```

man 2 sendto

- **sendto** può essere utilizzata sempre perché richiede di specificare l'indirizzo di destinazione
 - vedremo che esiste una variante **send** che può essere utilizzata solo se **socket** è stata connessa;
- restituisce **-1** in caso di errore oppure il numero di byte effettivamente trasmessi
- **flags** si può lasciare a zero

recv e recvfrom()

```
#include <sys/types.h>
#include <sys/socket.h>
```

man 2 recv

```
int recv(int s,
         void *buf, int len,
         unsigned int flags);
int recvfrom(int s, void *buf, int len,
             unsigned int flags,
             struct sockaddr *from,
             int *fromlen);
```

- ricevono in `buf` non più di `len` byte. Se `from` non è `NULL`, la struttura `sockaddr` verrà riempita con l'indirizzo del mittente
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente ricevuti
- `flags` si può lasciare a zero

recvfrom()

```
#include <sys/types.h>
#include <sys/socket.h>
int recvfrom(int s, void *buf, int len,
             unsigned int flags,
             struct sockaddr *from,
             int *fromlen);
```

man 2 recv

- ricevono in `buf` non più di `len` byte. Se `from` non è `NULL`, la struttura `sockaddr` verrà riempita con l'indirizzo del mittente
- restituisce `-1` in caso di errore oppure il numero di byte effettivamente ricevuti
- se non arriva alcun messaggio, il programma rimane sospeso (la chiamata è "bloccante")
- `flags` si può lasciare a zero

Esempio

C2) Creazione del socket dal lato client: il socket non abbisogna di un nome, quindi alla `socket` non viene fatta seguire una `bind`. L'argomento *protocol* può essere posto a `0` in quanto i due precedenti individuano univocamente il protocollo

```
sockfd = socket(PF_UNIX, SOCK_STREAM, 0);
```

C3) Connessione del socket locale al socket del server: si presuppone che il client conosca il nome del server-socket. Si noti il casting alla struttura di indirizzi generica nella chiamata `connect`

```
address.sun_family = AF_UNIX;
strcpy(address.sun_path, "server_socket");
len = sizeof(address);

result = connect(sockfd, (struct sockaddr *)&address, len);
if(result == -1) {
    perror("client");
    exit(1);
}
```


Esempio

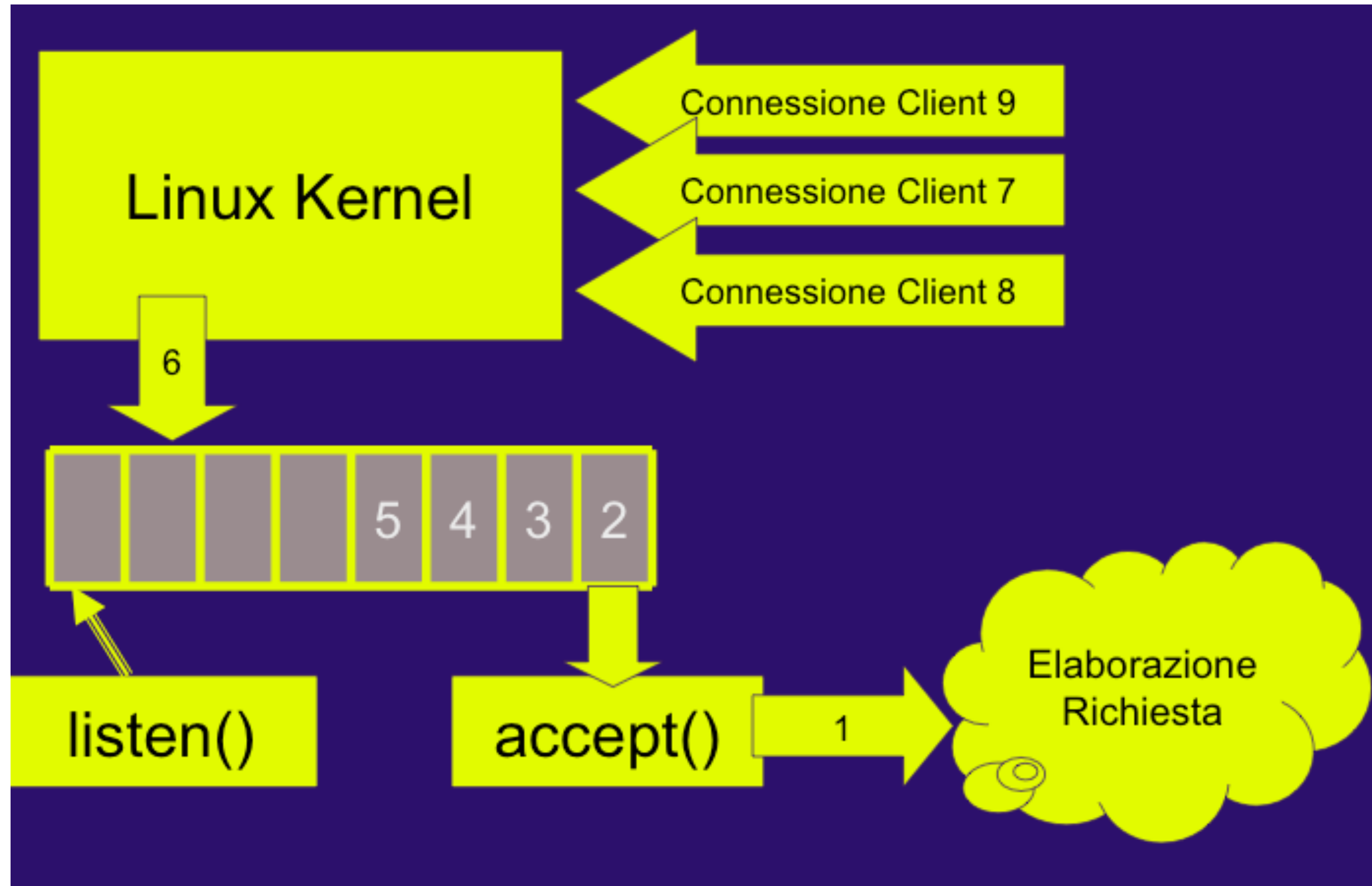
C4) trasmissione del carattere e ricezione della risposta dal server

```
    write(sockfd, &ch, 1);  
    read(sockfd, &ch, 1);  
    printf("char from server = %c\n", ch);  
    close(sockfd);  
    exit(0);  
}
```

Server a Programmazione Concorrente

- Un generico server attende le richieste di connessione su una determinata porta
- Possono arrivare richieste concorrenti e da molteplici client
- Una soluzione senza programmazione concorrente:
 - client serviti uno alla volta, finché una connessione non è terminata, non vengono serviti altri client interessati al servizio
 - N.B. comunque è il SO che gestisce le richieste concorrenti ed in effetti le serializza

La Coda di Connessione

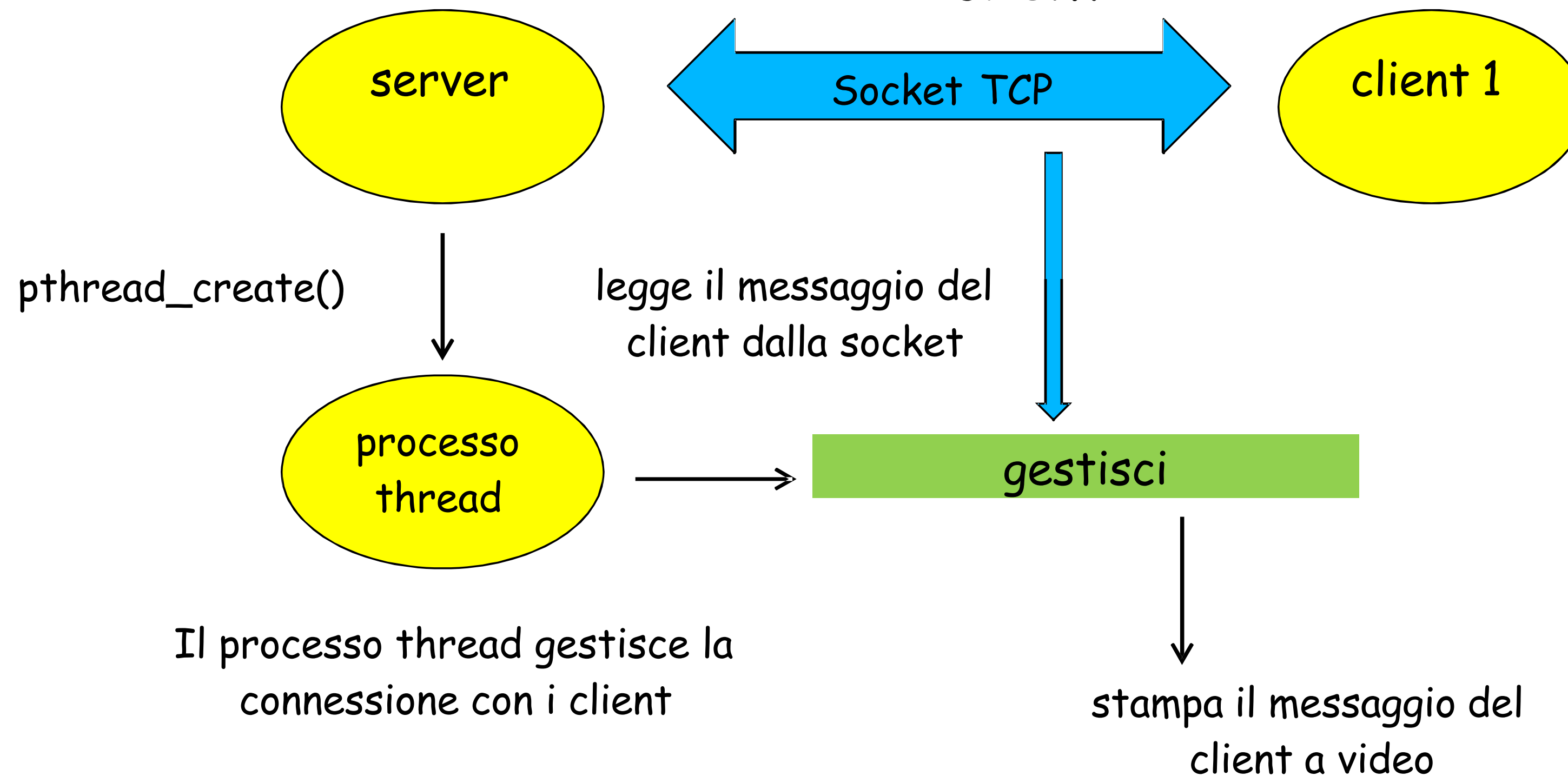


Server a Programmazione Concorrente

- Un server che gestisce sequenzialmente i vari client è insoddisfacente
 - il tempo di attesa di ogni client
 - potrebbe risultare eccessivo
 - dipende dalla durata delle altre connessioni
 - se le connessioni durano molto, il server ben presto diverrebbe indisponibile anche solo ad accodare le nuove richieste di connessioni
- Nei server reali le richieste di vari client devono essere gestite concorrentemente

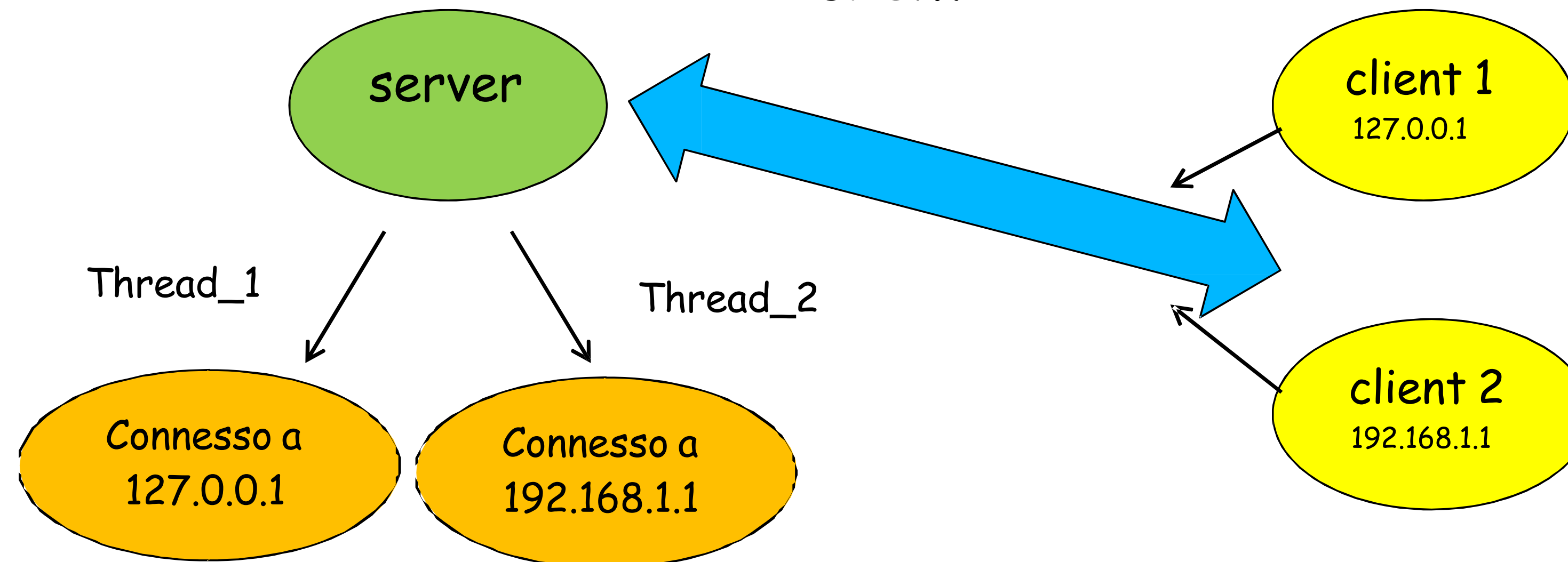
- Comunicazione client/server con socket TCP -

Schema Server concorrente
multi-thread per gestire più
client



- Comunicazione client/server con socket TCP -

Schema Server concorrente
multi-thread per gestire più
client



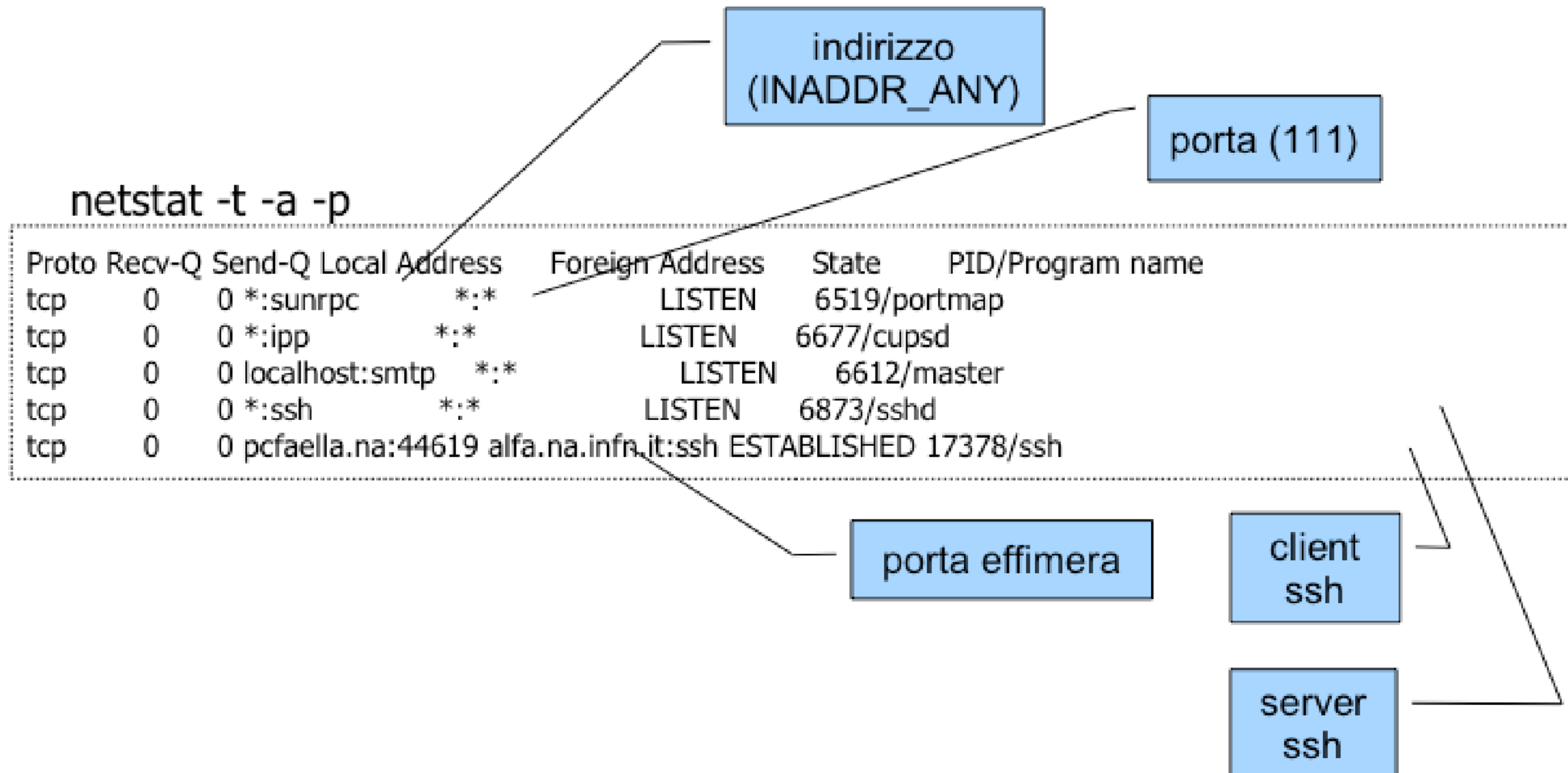
Risoluzione Indirizzi

- `gethostbyname()`
 - ▣ dato un hostname, restituisce una struttura dati che specifica anche i suoi indirizzi IP
- `gethostbyaddr()`
 - ▣ dato un indirizzo IP, restituisce una struttura dati che specifica anche il suo hostname
- `getservbyname()`
 - ▣ dato un nome di servizio e di protocollo, restituisce una struttura dati che specifica i suoi nomi e l'indirizzo di porta
- `gethostname()/getdomainname()`
 - ▣ restituiscono l'hostname della macchina
- `herror()`
 - ▣ stampa un messaggio di errore per `gethostname()`

Comandi utili: Netstat

- `netstat [-t] [-all] [-p] [-n]`
 - elenca tutti i socket di rete del sistema (non riguarda i socket locali)
 - “-t” mostra solo i socket TCP, cioè quelli con famiglia=PF_INET e tipo=SOCK_STREAM
 - “-all” (oppure “-a”) mostra anche i socket in ascolto
 - “-p” specifica il pid del processo che ha creato ciascun socket
 - “-n” mostra gli indirizzi e le porte in formato numerico (invece di simbolico)
 - ad es., `netstat -t -a -p -n` mostra tutti i socket TCP aperti, indicando il PID del processo corrispondente e mostrando gli indirizzi in formato numerico

Comandi utili: Netstat



Altri comandi utili

- `/sbin/ifconfig`
 - mostra l'indirizzo IP della macchina corrente
- `nslookup <nome di dominio>`
 - fornisce l'indirizzo IP di un host del quale conosciamo il nome
 - esempio: “`nslookup www.unina.it`”



Università degli Studi di Napoli Federico II

THANK YOU FOR YOUR ATTENTION

TRUST ME ...
I AM A ROBOT!

